CSC 255

Unit 10 Programming Assignment 2 [60 points]

Matrix Multiplication

First, we will review how to multiply two $n \times n$ matrices. For an example, let's multiply the following 2×2 matrices using the generic method of matrix multiplication.

$$\begin{bmatrix} 4 & 6 \\ 7 & -1 \end{bmatrix} \times \begin{bmatrix} 2 & 5 \\ 4 & -8 \end{bmatrix}$$

$$= \begin{bmatrix} 4 \times 2 + 6 \times 4 & 4 \times 5 + 6 \times -8 \\ 7 \times 2 + -1 \times 4 & 7 \times 5 + -1 \times -8 \end{bmatrix}$$

$$= \begin{bmatrix} 32 & -28 \\ 10 & 43 \end{bmatrix}$$

Let's also multiply the same two matrices using the recursive matrix multiply algorithm as identified in the **Introduction to Algorithms* text. The recursive matrix multiply algorithm requires eight recursive calls each of which represent eight sub-products.

Next, we can multiply the two matrices using **Strassen's Algorithm which is similar to the recursive matrix multiply algorithm with a crafty change that improves the runtime efficiency to $O(n^{2.8})$. Strassen's Algorithm requires seven recursive calls which is where the improvement stems from.

In order to effectively use a multithreaded approach to multiply two matrices, we must find a way to eliminate any race conditions among the separate threads used in making calculations concurrently. The idea is to split up the work required to compute the product of two matrices in a way that allows each thread to produce results that do not depend on the results of the other threads. This requires the recursive, divide-and-conquer approach to multiply the matrices. For this approach, we will implement concurrency using the recursive matrix multiplication algorithm and Strassen's Algorithm to find the product of two matrices. We can implement these algorithms using concurrency as described in the ***Introduction to Algorithms* text. Whenever we use a divide-and-conquer approach to implementing an algorithm, we will be able to integrate parallelization—in this case the parallelization scheme is concurrency using multiple threads.

Parallelized code using concurrency allows for multiple calculations to be concurrently performed on multiple threads of the CPU—just think of multiple CPUs doing work at the same time to produce a result. This approach of parallelism would likely generate the product faster than a typical serial approach (i.e., one thread of the CPU at any moment of time). Of course, unless we attempt this technique, we will not know for sure whether or not the parallel scheme will outperform the serial scheme.

Write a program that computes the product of two $n \times n$ matrices using the generic algorithm **[5 points]**, the recursive algorithm **[5 points]**, and Strassen's Algorithm without using the multithreaded approach **[10 points]**. Then implement the recursive matrix multiplication algorithm and Strassen's Algorithm using the multithreaded approach **[10 points each; 20 points]**.

Calculate the machine runtimes for all three serial approaches and capture the data in a tabular report **[5 points]**. Also, calculate the machine runtimes for the two parallel approaches and capture the data in a tabular report **[5 points]**.

In your report, include answers to the following questions **[2 points each; 10 points]**:

1) What values of n do we see any improvement in runtime using the parallel approach compared to the serial approach?
2) Why does the generic algorithm outperform the recursive algorithm in most cases?
3) What values of n allow for Strassen's Algorithm to outperform the recursive algorithm?
4) Why does the parallel recursive matrix multiplication algorithm use temporary matrix D?
5) Explain in simple terms how the recursive matrix multiplication algorithm works.

*A Simple Divide and Conquer Algorithm, p. 81 of MIT algorithms text.

**Strassen's Algorithm for Matrix Multiplication, p. 87 of MIT Algorithms text.

***A Parallel Divide-and-Conquer Algorithm for Matrix Multiplication, p. 771; Parallelizing Strassen's Method, p. 773 of MIT algorithms text.

For each of the multiplication algorithms below, assume the two matrices A and B are n x n matrices and the n x n matrix C is initialized to zeros. Each of the algorithms below will compute the product of A and B and assign the result to C. If memory is dynamically allocated when implementing any of these algorithms, ensure you deallocate appropriately to eliminate any memory leaks.

Pseudocode for Generic Matrix Multiplication

```
MATRIX_MULTIPLY(A, B, C, n)
    for i = 0 to n-1
        for j = 0 to n-1
            for k = 0 to n-1
                C[i][j] += A[i][k] * B[k][j]
```

Pseudocode for Recursive Matrix Multiplication Algorithm

```
RECURSIVE_MATRIX_MULTIPLY(A, B, C, n)
    if n == 1
        --Base case: multiply single elements
        C[0][0] = C[0][0] + A[0][0] * B[0][0]
```

```
else
    --Initialize result matrix C to zeros
    for i = 0 to n-1
        for j = 0 to n-1
            C[i][j] = 0

    --Partition matrices A, B, and C into submatrices of size n/2
    A11, A12, A21, A22
    B11, B12, B21, B22
    C11, C12, C21, C22

    --Recursive calls to multiply submatrices
    RECURSIVE_MATRIX_MULTIPLY(A11, B11, C11, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A11, B12, C12, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A21, B11, C21, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A21, B12, C22, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A12, B21, C11, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A12, B22, C12, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A22, B21, C21, n/2)
    RECURSIVE_MATRIX_MULTIPLY(A22, B22, C22, n/2)

    --Combine results into matrix C
    for i = 0 to n/2
        for j from 0 to n/2
            C[i][j] = C11[i][j]
            C[i][j + n/2] = C12[i][j]
            C[i + n/2][j] = C21[i][j]
            C[i + n/2][j + n/2] = C22[i][j]
```

Pseudocode for Strassen's Algorithm

```
STRASSENS_MATRIX_MULTIPLY(A, B, C, n)
    if n == 1
        --Base case: multiply single elements
        C[0][0] = C[0][0] + A[0][0] * B[0][0]
    else
        --Partition matrices A, B, and C into submatrices of size n/2
        A11, A12, A21, A22
        B11, B12, B21, B22
        C11, C12, C21, C22
        --Initialize S1 to S10 and P1 to P7
        S1, S2, S3, S4, S5, S6, S7, S8, S9, S10
        P1, P2, P3, P4, P5, P6, P7
        --Compute S1 to S10
        S1 = B12 - B22
        S2 = A11 + A12
        S3 = A21 + A22
        S4 = B21 - B11
        S5 = A11 + A22
        S6 = B11 + B22
        S7 = A12 – A22
        S8 = B21 + B22
        S9 = A11 - A21
        S10 = B11 + B12
        --Compute P1 to P7 recursively
        STRASSENS_MATRIX_MULTIPLY(A11, S1, P1, n/2)
        STRASSENS_MATRIX_MULTIPLY(S2, B22, P2, n/2)
        STRASSENS_MATRIX_MULTIPLY(S3, B11, P3, n/2)
        STRASSENS_MATRIX_MULTIPLY(A22, S4, P4, n/2)
```

```
        STRASSENS_MATRIX_MULTIPLY(S5, S6, P5, n/2)
        STRASSENS_MATRIX_MULTIPLY(S7, S8, P6, n/2)
        STRASSENS_MATRIX_MULTIPLY(S9, S10, P7, n/2)
        --Combine results into matrix C
        for i = 0 to n/2
            for j = 0 to n/2
                C[i][j] = C[i][j] + P5[i][j] + P4[i][j] - P2[i][j]
                          + P6[i][j]
                 C[i][j + newSize] = C[i][j + newSize] + P1[i][j] + P2[i][j]
                 C[i + newSize][j] = C[i + newSize][j] + P3[i][j] + P4[i][j]
                 C[i + newSize][j + newSize] = C[i + newSize][j + newSize]
                                               + P5[i][j] + P1[i][j]
                                               - P3[i][j] - P7[i][j]
```

Pseudocode for Parallel Recursive Matrix Multiplication Algorithm

```
PARALLEL_MATRIX_MULTIPLY(A, B, C, n)
    RECURSIVE_MATRIX_MULTIPLY(A, B, C, n, 0, 0, 0, 0, 0, 0)

RECURSIVE_MATRIX_MULTIPLY(A, B, C, n, startRowA, startColA, startRowB,
                          startColB, startRowC, startColC)
    if n == 1
        C[startRowC][startColC] = C[startRowC][startColC] +
                                  A[startRowA][startColA] *
                                  B[startRowB][startColB]
        return
    newSize = n / 2

    --Parallelize the creation and initialization of temporary matrix D of
    --size n x n.
```

```
parallel for i = 0 to n
    parallel for j = 0 to n
        D[i][j] = 0
--Fork 8 threads T1 to T8 for calculation of the products of the
--submatrices.
T1(RECURSIVE_MATRIX_MULTIPLY, A, B, C, newSize, startRowA, startColA,
   startRowB, startColB, startRowC, startColC)
T2(RECURSIVE_MATRIX_MULTIPLY, A, B, C, newSize, startRowA, startColA,
   startRowB, startColB + newSize, startRowC, startColC + newSize)
T3(RECURSIVE_MATRIX_MULTIPLY, A, B, C, newSize, startRowA + newSize,
   startColA, startRowB, startColB, startRowC + newSize, startColC)
T4(RECURSIVE_MATRIX_MULTIPLY, A, B, C, newSize, startRowA + newSize,
   startColA, startRowB, startColB + newSize, startRowC + newSize,
   startColC + newSize)
T5(RECURSIVE_MATRIX_MULTIPLY, A, B, D, newSize, startRowA, startColA +
   newSize, startRowB + newSize, startColB, 0, 0)
T6(RECURSIVE_MATRIX_MULTIPLY, A, B, D, newSize, startRowA, startColA +
   newSize, startRowB + newSize, startColB + newSize, 0, newSize)
T7(RECURSIVE_MATRIX_MULTIPLY, A, B, D, newSize, startRowA + newSize,
   startColA + newSize, startRowB + newSize, startColB, newSize, 0)
T8(RECURSIVE_MATRIX_MULTIPLY, A, B, D, newSize, startRowA + newSize,
   startColA + newSize, startRowB + newSize, startColB + newSize,
   newSize, newSize)
--Join the 8 threads.
T1.join()
T2.join()
T3.join()
T4.join()
T5.join()
T6.join()
```

```
    T7.join()
    T8.join()
    --Parallelize and combine results from D into C
    parallel for i = 0 to n
        parallel for j = 0 to n
            C[startRowC + i][startColC + j] = C[startRowC + i][startColC + j]
                                              + D[i][j]
```

Pseudocode for Parallel Strassen's Matrix Multiplication Algorithm

```
PARALLEL_MATRIX_MULTIPLY(A, B, C, n)
    STRASSEN_MATRIX_MULTIPLY(A, B, C, n, 0, 0, 0, 0, 0, 0)

STRASSEN_MATRIX_MULTIPLY(A, B, C, n, startRowA, startColA, startRowB,
                         startColB, startRowC, startColC)
    if n == 1
        C[startRowC][startColC] = C[startRowC][startColC] +
                                  A[startRowA][startColA] *
                                  B[startRowB][startColB]
        return
    newSize = n / 2
    --Create temporary matrices S1 to S10 and P1 to P7 of size newSize x
    --newSize.
    S1, S2, S3, S4, S5, S6, S7, S8, S9, S10
    P1, P2, P3, P4, P5, P6, P7
    --Parallelize the initialization of temporary matrices S1 to S10
    --and P1 to P7 of size newSize x newSize.
    parallel for i = 0 to newSize
        parallel for j = 0 to newSize
            S1[i][j] = 0, S2[i][j] = 0, S3[i][j] = 0, S4[i][j] = 0,
```

```
        S5[i][j] = 0, S6[i][j] = 0, S7[i][j] = 0, S8[i][j] = 0,
        S9[i][j] = 0, S10[i][j] = 0, P1[i][j] = 0, P2[i][j] = 0,
        P3[i][j] = 0, P4[i][j] = 0, P5[i][j] = 0, P6[i][j] = 0,
        P7[i][j] = 0

--Fork 7 threads T1 to T8 for calculation of the products of the
--submatrices.
--Compute S1 to S10
--S1 = B12 – B22
subtractMatrix(B, startRowB, startColB + newSize, startRowB + newSize,
startColB + newSize, S1, newSize)
--S2 = A11 + A12
addMatrix(A, startRowA, startColA, startRowA, startColA + newSize, S2,
newSize)
--S3 = A21 + A22
addMatrix(A, startRowA + newSize, startColA, startRowA + newSize,
startColA + newSize, S3, newSize)
--S4 = B21 – B11
subtractMatrix(B, startRowB + newSize, startColB, startRowB, startColB,
S4, newSize)
--S5 = A11 + A22
addMatrix(A, startRowA, startColA, startRowA + newSize, startColA +
newSize, S5, newSize)
--S6 = B11 + B22
addMatrix(B, startRowB, startColB, startRowB + newSize, startColB +
newSize, S6, newSize)
--S7 = A12 – A22
subtractMatrix(A, startRowA, startColA + newSize, startRowA + newSize,
startColA + newSize, S7, newSize)
```

```
--S8 = B21 + B22
addMatrix(B, startRowB + newSize, startColB, startRowB + newSize,
startColB + newSize, S8, newSize)
--S9 = A11 – A21
subtractMatrix(A, startRowA, startColA, startRowA + newSize, startColA,
S9, newSize)
--S10 = B11 + B12
addMatrix(B, startRowB, startColB, startRowB, startColB + newSize,
S10, newSize)

--Fork 7 threads T1 to T8 for calculation of the products of the
--submatrices.
--P1 = A11*B12 - A11*B22 = A11*S1
T1(STRASSEN_MATRIX_MULTIPLY, A, S1, P1, newSize, startRowA, startColA,
   0, 0, 0, 0)
--P2 = A11*B22 + A12*B22 = S2*B22
T2(STRASSEN_MATRIX_MULTIPLY, S2, B, P2, newSize, 0, 0, startRowB +
   newSize, startColB + newSize, 0, 0)
--P3 = A21*B11 + A22*B11 = S3*B11
T3(STRASSEN_MATRIX_MULTIPLY, S3, B, P3, newSize, 0, 0, startRowB,
   startColB, 0, 0)
--P4 = A22*B21 - A22*B11 = A22*S4
T4(STRASSEN_MATRIX_MULTIPLY, A, S4, P4, newSize, startRowA + newSize,
   startColA + newSize, 0, 0, 0, 0)
--P5 = A11*B11 + A11*B22 + A22*B11 + A22*B22 = S5*S6
T5(STRASSEN_MATRIX_MULTIPLY, S5, S6, P5, newSize, 0, 0, 0, 0, 0, 0)
--P6 = A12*B21 + A12*B22 - A22*B21 - A22*B22 = S7*S8
T6(STRASSEN_MATRIX_MULTIPLY, S7, S8, P6, newSize, 0, 0, 0, 0, 0, 0)
--P7 = A11*B11 + A11*B12 - A21*B11 - A21*B12 = S9*S10
```

```
T7(STRASSEN_MATRIX_MULTIPLY, S9, S10, P7, newSize, 0, 0, 0, 0, 0, 0)
--Join the 7 threads.
T1.join()
T2.join()
T3.join()
T4.join()
T5.join()
T6.join()
T7.join()

--Update Cij
--C11 = C11 + P5 + P4 - P2 + P6
parallel for i = 0 to newSize
    parallel for j = 0 to newSize
        C[i][j] = C[i][j] + P5[i][j] + P4[i][j] - P2[i][j] +
                  P6[i][j]
--C12 = C12 + P1 + P2
parallel for i = 0 to newSize
    parallel for j = 0 to newSize
        C[i][j + newSize] = C[i][j + newSize] + P1[i][j] + P2[i][j]
--C21 = C21 + P3 + P4
parallel for i = 0 to newSize
    parallel for j = 0 to newSize
        C[i + newSize][j] = C[i + newSize][j] + P3[i][j] + P4[i][j]
--C22 = C22 + P5 + P1 - P3 - P7
parallel for i = 0 to newSize
    parallel for j = 0 to newSize
        C[i + newSize][j + newSize] = C[i + newSize][j + newSize] +
                                      P5[i][j] + P1[i][j] - P3[i][j] –
                                      P7[i][j]
```